보안 모범사례 가이드

개요

CometBFT 기반 블록체인 네트워크의 보안은 다층적인 접근 방식을 필요로 합니다. 이 가이드는 알려진 보안 취약점과 대응 방법, 보안 감사 체크리스트, 그리고 운영 환경에서의 보안 모범사례를 제공합니다.

보안은 단순히 기술적인 구현만의 문제가 아니라, 운영 프로세스, 인적 요소, 물리적 보안까지 포괄하는 종합적인 접근이 필요합니다. 이 문서는 CometBFT 네트워크를 안전하게 운영하기 위한 실무적인 지침을 제공합니다.

핵심 보안 원칙

심층 방어 (Defense in Depth)

다층적 보안 체계를 구축하여 단일 실패점을 방지합니다:

1. **네트워크 레벨**: 방화벽, VPN, 네트워크 분할
2. **시스템 레벨**: OS 보안, 접근 제어, 모니터링
3. **애플리케이션 레벨**: 입력 검증, 암호화, 인증
4. **데이터 레벨**: 암호화, 백업, 무결성 검증

최소 권한 원칙 (Principle of Least Privilege)

각 구성 요소와 사용자에게 필요한 최소한의 권한만 부여합니다:

```
# 예시: 시스템 사용자 권한 분리

cometbft_user:
```

```
- 읽기: /opt/cometbft/config/
- 쓰기: /opt/cometbft/data/
- 실행: /opt/cometbft/bin/cometbft

validator_user:
- 읽기: /opt/validator/config/
- 쓰기: /opt/validator/data/
- 실행: /opt/validator/bin/signer
```

실패 시 안전 (Fail-Safe)

시스템 실패 시 보안이 유지되도록 설계합니다:

- 네트워크 연결 실패 시 자동으로 안전 모드로 전환
- 키 관리 시스템 오류 시 서명 중단
- 비정상적인 활동 감지 시 알림 및 차단

알려진 보안 취약점과 대응 방법

1. 이중 서명 공격 (Double Signing Attack)

취약점 설명

악의적인 밸리데이터가 동일한 높이에서 서로 다른 블록에 서명하여 네트워크를 분할시키는 공격입니다.

대응 방법

```go
// HRS (Height-Round-Step) 검증 강화
func (lss *FilePVLastSignState) CheckHRS(height  int64, round  int32, step  int8
// 높이 회귀 방지
if  lss.Height > height {
return false, fmt.Errorf("height regression. Got %v, last height %v", height, ls
}
// 라운드 회귀 방지
if  lss.Height == height && lss.Round > round {
return false, fmt.Errorf("round regression at height %v. Got %v, last round %v",
height, round, lss.Round)
}
// 단계 회귀 방지
if  lss.Height == height && lss.Round == round && lss.Step > step {
return false, fmt.Errorf("step regression at height %v round %v. Got %v, last st
height, round, step, lss.Step)
}
return true, nil
}
```

모니터링 및 탐지

```go
// 이중 서명 탐지 시스템
type DoubleSignDetector struct {
```

```go
signatures  map[string]*SignatureRecord
alerts  chan Alert
}

type SignatureRecord struct {
Height  int64
Round  int32
BlockHash []byte
Signature []byte
Timestamp time.Time
}

func (dsd *DoubleSignDetector) CheckSignature(height  int64, round  int32,
blockHash []byte, signature []byte,
validatorAddr []byte) error {
key := fmt.Sprintf("%X-%d-%d", validatorAddr, height, round)
if  existing, found := dsd.signatures[key];  found {
if !bytes.Equal(existing.BlockHash, blockHash) {
// 이중 서명 탐지!
alert := Alert{
Type: "DOUBLE_SIGNING",
Validator: validatorAddr,
Height: height,
```

```
Round: round,
Evidence: map[string]interface{}{
"first_block": existing.BlockHash,
"second_block": blockHash,
},
}
dsd.alerts <- alert
return  fmt.Errorf("double signing detected for validator %X at height %d round
validatorAddr, height, round)
}
} else {
dsd.signatures[key] = &SignatureRecord{
Height: height,
Round: round,
BlockHash: blockHash,
Signature: signature,
Timestamp: time.Now(),
}
}
return  nil
}
```

2. 장거리 공격 (Long Range Attack)

취약점 설명

공격자가 과거의 밸리데이터 키를 획득하여 체인의 초기 상태부터 대체 체인을 생성하는 공격입니다.

대응 방법

```
// 체크포인트 기반 검증
type CheckpointValidator struct {
trustedCheckpoints  map[int64]CheckpointData
maxLookback  int64
}

type CheckpointData struct {
Height  int64
BlockHash []byte
ValidatorSet *ValidatorSet
Timestamp time.Time
}

func (cv *CheckpointValidator) ValidateBlock(block *Block) error {
// 신뢰할 수 있는 체크포인트와의 거리 확인
nearestCheckpoint := cv.findNearestCheckpoint(block.Height)
if  block.Height - nearestCheckpoint.Height > cv.maxLookback {
return  fmt.Errorf("block too far from trusted checkpoint: %d blocks",
block.Height - nearestCheckpoint.Height)
```

```
}

// 체크포인트로부터의 체인 검증

return  cv.validateChainFromCheckpoint(nearestCheckpoint, block)

}



// 소셜 합의를 통한 체크포인트 설정

func (cv *CheckpointValidator) AddTrustedCheckpoint(height  int64, blockHash []b

valSet *ValidatorSet) error {

// 다수의 독립적인 소스에서 검증된 체크포인트만 추가

checkpoint := CheckpointData{

Height: height,

BlockHash: blockHash,

ValidatorSet: valSet,

Timestamp: time.Now(),

}

cv.trustedCheckpoints[height] = checkpoint

return  nil

}
```

3. 네트워크 분할 공격 (Network Partition Attack)

취약점 설명

네트워크를 분할하여 서로 다른 부분이 독립적으로 합의에 도달하도록 하는 공격입니다.

대응 방법

```go
// 네트워크 건강성 모니터링
type NetworkHealthMonitor struct {
peerConnections  map[string]*PeerConnection
minPeerCount  int
alertThreshold  float64
}


func (nhm *NetworkHealthMonitor) CheckNetworkHealth() error {
activePeers := 0
totalPeers := len(nhm.peerConnections)
for  _, conn := range  nhm.peerConnections {
if  conn.IsActive() && conn.LastSeen.After(time.Now().Add(-30*time.Second)) {
activePeers++
}
}
healthRatio := float64(activePeers) / float64(totalPeers)
if  activePeers < nhm.minPeerCount || healthRatio < nhm.alertThreshold {
return  fmt.Errorf("network health degraded: %d/%d peers active (%.2f%%)",
activePeers, totalPeers, healthRatio*100)
}
return  nil
}
```

```go
// 다중 네트워크 경로 설정
func (nhm *NetworkHealthMonitor) ConfigureRedundantPaths() {
// 여러 ISP를 통한 연결
// VPN 터널을 통한 백업 연결
// 위성 연결 등 대체 통신 수단
}
```

4. 타이밍 공격 (Timing Attack)

취약점 설명

서명 검증 시간의 차이를 이용하여 개인키 정보를 추출하는 공격입니다.

대응 방법

```go
// 상수 시간 비교 함수
func constantTimeCompare(a, b []byte) bool {
if len(a) != len(b) {
return false
}
var result byte
for i := 0; i < len(a); i++ {
result |= a[i] ^ b[i]
}
return result == 0
```

```go
}

// 서명 검증에서 타이밍 공격 방지
func (pv *FilePV) verifySignatureConstantTime(msg []byte, sig []byte) bool {
// 항상 동일한 시간이 소요되도록 구현
expectedSig, err := pv.Key.PrivKey.Sign(msg)
if  err != nil {
// 오류 발생 시에도 동일한 시간 소요
time.Sleep(time.Microsecond * 100)
return false
}
return  constantTimeCompare(sig, expectedSig)
}

// 서명 검증 시간 정규화
type TimingNormalizer struct {
baseDelay time.Duration
jitter time.Duration
}

func (tn *TimingNormalizer) NormalizeOperation(operation  func() bool) bool {
start := time.Now()
result := operation()
```

```
elapsed := time.Since(start)

// 최소 지연 시간 보장

if elapsed < tn.baseDelay {

time.Sleep(tn.baseDelay - elapsed)

}

// 랜덤 지터 추가

jitter := time.Duration(rand.Int63n(int64(tn.jitter)))

time.Sleep(jitter)

return result

}
```

5. 사이드 채널 공격 (Side Channel Attack)

취약점 설명

전력 소비, 전자기 방출, 음향 등의 부수적인 정보를 통해 암호화 키를 추출하는 공격입니다.

대응 방법

```
// 메모리 보호 및 정리

type SecureMemory struct {

data []byte

size int

}
```

```go
func NewSecureMemory(size int) *SecureMemory {
// 메모리 잠금으로 스왑 방지
data := make([]byte, size)
if err := mlock(data); err != nil {
panic(fmt.Sprintf("Failed to lock memory: %v", err))
}
return &SecureMemory{
data: data,
size: size,
}
}

func (sm *SecureMemory) Clear() {
// 메모리를 안전하게 지우기
for i := range sm.data {
sm.data[i] = 0
}
// 컴파일러 최적화 방지
runtime.KeepAlive(sm.data)
}

func (sm *SecureMemory) Destroy() {
sm.Clear()
```

```go
munlock(sm.data)
}

// 키 사용 후 즉시 정리
func (pv *FilePV) signWithSecureCleanup(msg []byte) ([]byte, error) {
// 임시 키 복사본 생성
keyData := NewSecureMemory(len(pv.Key.PrivKey.Bytes()))
copy(keyData.data, pv.Key.PrivKey.Bytes())
defer keyData.Destroy() // 사용 후 즉시 정리
// 서명 생성
signature, err := pv.Key.PrivKey.Sign(msg)
if err != nil {
return nil, err
}
return signature, nil
}
```

설정 보안 강화

1. 네트워크 설정

```toml
# config.toml - 보안 강화 설정
```

```
[p2p]

# 외부 주소 명시적 설정

external_address = "your-public-ip:26656"



# 최대 연결 수 제한

max_num_inbound_peers = 40

max_num_outbound_peers = 10



# 프라이빗 피어 설정 (센티넬 노드)

private_peer_ids = "peer1_id,peer2_id"



# 중복 IP 연결 방지

allow_duplicate_ip = false



# 피어 교환 비활성화 (프라이빗 네트워크)

pex = false



# 패킷 크기 제한

max_packet_msg_payload_size = 1024



# 전송 속도 제한

send_rate = 5120000 # 5MB/s

recv_rate = 5120000 # 5MB/s
```

```
[rpc]

# RPC 서버 로컬 바인딩

laddr = "tcp://127.0.0.1:26657"



# 위험한 RPC 비활성화

unsafe = false



# 연결 수 제한

max_open_connections = 100



# 요청 크기 제한

max_body_bytes = 1000000 # 1MB

max_header_bytes = 1048576 # 1MB



# TLS 활성화

tls_cert_file = "server.crt"

tls_key_file = "server.key"



[mempool]

# 메모리풀 크기 제한

size = 5000

max_txs_bytes = 1073741824 # 1GB
```

```
# 트랜잭션 크기 제한
max_tx_bytes = 1048576 # 1MB


[consensus]
# 타임아웃 설정
timeout_propose = "3s"
timeout_propose_delta = "500ms"
timeout_prevote = "1s"
timeout_prevote_delta = "500ms"
timeout_precommit = "1s"
timeout_precommit_delta = "500ms"


# 빈 블록 생성 비활성화 (선택적)
create_empty_blocks = false
create_empty_blocks_interval = "0s"
```

2. 방화벽 설정

```
#!/bin/bash
# 방화벽 규칙 설정


# 기본 정책: 모든 연결 차단
```

```
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT

# 로컬 루프백 허용
iptables -A INPUT -i lo -j ACCEPT

# 기존 연결 유지
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# SSH 접근 허용 (특정 IP만)
iptables -A INPUT -p tcp --dport 22 -s YOUR_ADMIN_IP -j ACCEPT

# CometBFT P2P 포트 (26656)
iptables -A INPUT -p tcp --dport 26656 -j ACCEPT

# CometBFT RPC 포트 (로컬만)
iptables -A INPUT -p tcp --dport 26657 -s 127.0.0.1 -j ACCEPT

# ABCI 포트 (로컬만)
iptables -A INPUT -p tcp --dport 26658 -s 127.0.0.1 -j ACCEPT

# DDoS 방지
```

```
iptables -A INPUT -p tcp --dport 26656 -m limit --limit 25/minute --l

# 규칙 저장
iptables-save > /etc/iptables/rules.v4
```

3. 시스템 보안 설정

```
#!/bin/bash
# 시스템 보안 강화

# 전용 사용자 생성
useradd -r -s /bin/false -d /opt/cometbft cometbft

# 디렉토리 권한 설정
mkdir -p /opt/cometbft/{config,data,logs}
chown -R cometbft:cometbft /opt/cometbft
chmod 700 /opt/cometbft/config
chmod 755 /opt/cometbft/data
chmod 755 /opt/cometbft/logs

# 키 파일 권한 설정
chmod 600 /opt/cometbft/config/priv_validator_key.json
chmod 600 /opt/cometbft/config/node_key.json
```

```
# 시스템 한계 설정
cat >> /etc/security/limits.conf << EOF
cometbft soft nofile 65536
cometbft hard nofile 65536
cometbft soft nproc 32768
cometbft hard nproc 32768
EOF

# 커널 매개변수 조정
cat >> /etc/sysctl.conf << EOF
# 네트워크 보안
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_max_syn_backlog = 2048
net.ipv4.tcp_synack_retries = 2
net.ipv4.tcp_syn_retries = 5

# 파일 디스크립터 한계
fs.file-max = 2097152

# 메모리 보호
kernel.dmesg_restrict = 1
kernel.kptr_restrict = 2
```

```
EOF


sysctl -p
```

키 관리 보안

1. 하드웨어 보안 모듈 (HSM) 통합

```
// HSM 인터페이스 구현
type HSMPrivValidator struct {
client HSMClient
keyID string
}


func NewHSMPrivValidator(endpoint, keyID string) (*HSMPrivValidator, error) {
client, err := NewHSMClient(endpoint)
if err != nil {
return nil, err
}
return &HSMPrivValidator{
client: client,
keyID: keyID,
}, nil
```

```
}

func (hsm *HSMPrivValidator) GetPubKey() (crypto.PubKey, error) {
pubKeyBytes, err := hsm.client.GetPublicKey(hsm.keyID)
if err != nil {
return nil, err
}
return ed25519.PubKey(pubKeyBytes), nil
}

func (hsm *HSMPrivValidator) SignVote(chainID string, vote *cmtproto.Vote, sign
signBytes := types.VoteSignBytes(chainID, vote)
signature, err := hsm.client.Sign(hsm.keyID, signBytes)
if err != nil {
return err
}
vote.Signature = signature
if signExtension && vote.Type == types.PrecommitType && !types.ProtoBlockIDIsNi
extSignBytes, nonRpExtSignBytes := types.VoteExtensionSignBytes(chainID, vote)
extSig, err := hsm.client.Sign(hsm.keyID, extSignBytes)
if err != nil {
return err
}
```

```go
nonRpExtSig, err := hsm.client.Sign(hsm.keyID, nonRpExtSignBytes)
if err != nil {
return err
}
vote.ExtensionSignature = extSig
vote.NonRpExtensionSignature = nonRpExtSig
}
return nil
}
```

2. 키 분할 및 임계값 서명

```go
// Shamir's Secret Sharing을 이용한 키 분할
type ThresholdSigner struct {
threshold int
shares map[int][]byte
pubKey crypto.PubKey
}

func NewThresholdSigner(threshold, total int, masterKey crypto.PrivKey) (*Thre
// 키를 threshold-of-total 방식으로 분할
shares, err := shamirSplit(masterKey.Bytes(), threshold, total)
```

```go
if  err != nil {
return  nil, err
}
return &ThresholdSigner{
threshold: threshold,
shares: shares,
pubKey: masterKey.PubKey(),
}, nil
}


func (ts *ThresholdSigner) Sign(msg []byte, availableShares  map[int][]byte) ([]
if  len(availableShares) < ts.threshold {
return  nil, fmt.Errorf("insufficient shares: need %d, got %d", ts.threshold, le
}
// 키 재구성
reconstructedKey, err := shamirReconstruct(availableShares, ts.threshold)
if  err != nil {
return  nil, err
}
// 서명 생성
privKey := ed25519.PrivKey(reconstructedKey)
signature, err := privKey.Sign(msg)
// 키 즉시 정리
```

```go
for i := range reconstructedKey {
reconstructedKey[i] = 0
}
return signature, err
}
```

3. 키 순환 (Key Rotation)

```go
// 자동 키 순환 시스템
type KeyRotationManager struct {
currentKey *FilePV
nextKey *FilePV
rotationEpoch int64
rotationHeight int64
}

func (krm *KeyRotationManager) ShouldRotate(currentHeight int64) bool {
return currentHeight >= krm.rotationHeight
}

func (krm *KeyRotationManager) PrepareRotation(newKeyPath, newStatePath string)
// 새로운 키 생성
newPV, err := GenFilePV(newKeyPath, newStatePath, nil)
```

```go
if err != nil {
return err
}
krm.nextKey = newPV
return nil
}


func (krm *KeyRotationManager) ExecuteRotation() error {
if krm.nextKey == nil {
return errors.New("next key not prepared")
}
// 이전 키 백업
backupPath := fmt.Sprintf("%s.backup.%d", krm.currentKey.Key.filePath, time.Now(
if err := BackupPrivValidator(krm.currentKey, backupPath); err != nil {
return err
}
// 키 교체
krm.currentKey = krm.nextKey
krm.nextKey = nil
krm.rotationEpoch++
return nil
}
```

모니터링 및 알림

1. 보안 이벤트 모니터링

```go
// 보안 이벤트 모니터링 시스템
type SecurityMonitor struct {
events  chan SecurityEvent
rules []SecurityRule
alerter Alerter
logger log.Logger
}

type SecurityEvent struct {
Type  string
Severity  string
Source  string
Message  string
Metadata  map[string]interface{}
Timestamp time.Time
}

type SecurityRule struct {
Name  string
```

```
Pattern  string

Severity  string

Action  string

Threshold  int

TimeWindow time.Duration

}


func (sm *SecurityMonitor) ProcessEvent(event SecurityEvent) {

sm.logger.Info("Security event",

"type", event.Type,

"severity", event.Severity,

"source", event.Source,

"message", event.Message)

// 규칙 매칭

for  _, rule := range  sm.rules {

if  sm.matchRule(rule, event) {

sm.handleRuleMatch(rule, event)

}

}

}


func (sm *SecurityMonitor) handleRuleMatch(rule SecurityRule, event SecurityEven

switch  rule.Action {
```

```
case "alert":
sm.alerter.SendAlert(Alert{
Title: fmt.Sprintf("Security Rule Triggered: %s", rule.Name),
Message: event.Message,
Severity: rule.Severity,
Source: event.Source,
})
case "block":
// IP 차단 또는 연결 종료
sm.blockSource(event.Source)
case "quarantine":
// 격리 모드로 전환
sm.enterQuarantineMode()
}
}


// 보안 메트릭 수집
type SecurityMetrics struct {
FailedSignAttempts prometheus.Counter
DoubleSignDetections prometheus.Counter
NetworkPartitions prometheus.Counter
UnauthorizedAccess prometheus.Counter
KeyRotations prometheus.Counter
```

```go
}


func (sm *SecurityMetrics) RecordFailedSignAttempt(validator  string) {
sm.FailedSignAttempts.With(prometheus.Labels{
"validator": validator,
}).Inc()
}
```

2. 실시간 알림 시스템

```go
// 다중 채널 알림 시스템
type MultiChannelAlerter struct {
channels []AlertChannel
}


type AlertChannel interface {
SendAlert(alert Alert) error
IsHealthy() bool
}


// Slack 알림
type SlackAlerter struct {
webhookURL  string
```

```go
channel  string
}

func (sa *SlackAlerter) SendAlert(alert Alert) error {
payload := map[string]interface{}{
"channel": sa.channel,
"text": fmt.Sprintf("🚨 *%s*\n%s", alert.Title, alert.Message),
"color": sa.getSeverityColor(alert.Severity),
}
return  sa.sendToSlack(payload)
}

// PagerDuty 알림
type PagerDutyAlerter struct {
integrationKey  string
client *pagerduty.Client
}

func (pda *PagerDutyAlerter) SendAlert(alert Alert) error {
event := pagerduty.Event{
Type: "trigger",
ServiceKey: pda.integrationKey,
Description: alert.Title,
```

```
Details: alert.Message,
Severity: alert.Severity,
}
return  pda.client.CreateEvent(event)
}

// 이메일 알림
type EmailAlerter struct {
smtpHost  string
smtpPort  int
username  string
password  string
recipients []string
}

func (ea *EmailAlerter) SendAlert(alert Alert) error {
subject := fmt.Sprintf("[CometBFT Security] %s", alert.Title)
body := fmt.Sprintf(`
보안 알림이 발생했습니다.

제목: %s
심각도: %s
소스: %s
```

```
시간: %s


내용:
%s


즉시 확인하시기 바랍니다.
`, alert.Title, alert.Severity, alert.Source, alert.Timestamp.Format(time.RFC333
return  ea.sendEmail(ea.recipients, subject, body)
}
```

보안 감사 체크리스트

1. 시스템 보안 점검

```
// 자동화된 보안 감사 시스템
type SecurityAuditor struct {
checks []SecurityCheck
logger log.Logger
}

type SecurityCheck interface {
Name() string
Description() string
```

```
Execute() (SecurityCheckResult, error)
}

type SecurityCheckResult struct {
Passed  bool
Score  int
Issues []string
Warnings []string
Recommendations []string
}

// 파일 권한 검사
type FilePermissionCheck struct {
files  map[string]os.FileMode
}

func (fpc *FilePermissionCheck) Execute() (SecurityCheckResult, error) {
result := SecurityCheckResult{Passed: true, Score: 100}
for  filePath, expectedMode := range  fpc.files {
info, err := os.Stat(filePath)
if  err != nil {
result.Issues = append(result.Issues, fmt.Sprintf("Cannot access file: %s", file
result.Passed = false
```

```
result.Score -= 20
continue
}
actualMode := info.Mode().Perm()
if  actualMode != expectedMode {
result.Issues = append(result.Issues,
fmt.Sprintf("Incorrect permissions for %s: got %o, expected %o",
filePath, actualMode, expectedMode))
result.Passed = false
result.Score -= 10
}
}
return  result, nil
}


// 네트워크 설정 검사
type NetworkConfigCheck struct {
config *Config
}


func (ncc *NetworkConfigCheck) Execute() (SecurityCheckResult, error) {
result := SecurityCheckResult{Passed: true, Score: 100}
// RPC 보안 검사
```

```
if  ncc.config.RPC.Unsafe {
result.Warnings = append(result.Warnings, "Unsafe RPC is enabled")
result.Score -= 15
}
if !ncc.config.RPC.IsTLSEnabled() {
result.Issues = append(result.Issues, "TLS is not enabled for RPC")
result.Passed = false
result.Score -= 25
}
// P2P 보안 검사
if  ncc.config.P2P.AllowDuplicateIP {
result.Warnings = append(result.Warnings, "Duplicate IP connections are allowed"
result.Score -= 10
}
if  ncc.config.P2P.MaxNumInboundPeers > 100 {
result.Warnings = append(result.Warnings, "High number of inbound peers may impa
result.Score -= 5
}
return  result, nil
}


// 키 보안 검사
type KeySecurityCheck struct {
```

```
keyPath  string

statePath  string

}



func (ksc *KeySecurityCheck) Execute() (SecurityCheckResult, error) {

result := SecurityCheckResult{Passed: true, Score: 100}

// 키 파일 존재 확인

if  _, err := os.Stat(ksc.keyPath);  os.IsNotExist(err) {

result.Issues = append(result.Issues, "Private key file does not exist")

result.Passed = false

result.Score = 0

return  result, nil

}

// 키 파일 권한 확인

keyInfo, err := os.Stat(ksc.keyPath)

if  err != nil {

result.Issues = append(result.Issues, "Cannot access private key file")

result.Passed = false

result.Score -= 50

} else  if  keyInfo.Mode().Perm() != 0600 {

result.Issues = append(result.Issues, "Private key file has incorrect permission

result.Passed = false

result.Score -= 30
```

```
}

// 상태 파일 권한 확인

stateInfo, err := os.Stat(ksc.statePath)

if err != nil {

result.Warnings = append(result.Warnings, "Cannot access state file")

result.Score -= 10

} else if stateInfo.Mode().Perm() != 0600 {

result.Issues = append(result.Issues, "State file has incorrect permissions")

result.Passed = false

result.Score -= 20

}

// 백업 존재 확인

backupPath := ksc.keyPath + ".backup"

if _, err := os.Stat(backupPath); os.IsNotExist(err) {

result.Warnings = append(result.Warnings, "No backup found for private key")

result.Score -= 15

}

return result, nil

}
```

2. 정기 보안 점검 스케줄

```
// 정기 보안 감사 스케줄러
```

```go
type SecurityAuditScheduler struct {
auditor *SecurityAuditor
schedule  map[string]time.Duration
results  chan SecurityAuditReport
ticker *time.Ticker
}

type SecurityAuditReport struct {
Timestamp time.Time
Results  map[string]SecurityCheckResult
OverallScore  int
Critical []string
Warnings []string
}

func (sas *SecurityAuditScheduler) Start() {
sas.ticker = time.NewTicker(24 * time.Hour) // 일일 감사
go  func() {
for  range  sas.ticker.C {
report := sas.runFullAudit()
sas.results <- report
if  report.OverallScore < 70 {
sas.sendCriticalAlert(report)
```

```
}
}
}()
}

func (sas *SecurityAuditScheduler) runFullAudit() SecurityAuditReport {
report := SecurityAuditReport{
Timestamp: time.Now(),
Results: make(map[string]SecurityCheckResult),
}
totalScore := 0
checkCount := 0
for _, check := range sas.auditor.checks {
result, err := check.Execute()
if err != nil {
result = SecurityCheckResult{
Passed: false,
Score: 0,
Issues: []string{fmt.Sprintf("Check failed: %v", err)},
}
}
report.Results[check.Name()] = result
totalScore += result.Score
```

```
checkCount++

// 중요한 이슈 수집

for _, issue := range result.Issues {

report.Critical = append(report.Critical, fmt.Sprintf("%s: %s", check.Name(), is

}

// 경고 수집

for _, warning := range result.Warnings {

report.Warnings = append(report.Warnings, fmt.Sprintf("%s: %s", check.Name(), wa

}

}

if checkCount > 0 {

report.OverallScore = totalScore / checkCount

}

return report

}
```

인시던트 대응

1. 보안 인시던트 대응 절차

```
// 인시던트 대응 시스템

type IncidentResponseSystem struct {

severity map[string]int
```

```
procedures  map[string]ResponseProcedure

responders []Responder

logger log.Logger

}


type ResponseProcedure struct {

Name  string

Steps []ResponseStep

Escalation []string

Automation  bool

}


type ResponseStep struct {

Action  string

Description  string

Timeout time.Duration

Required  bool

}


func (irs *IncidentResponseSystem) HandleIncident(incident SecurityIncident) err

irs.logger.Error("Security incident detected",

"type", incident.Type,

"severity", incident.Severity,
```

```
"description", incident.Description)

// 심각도에 따른 자동 대응

switch incident.Severity {

case "CRITICAL":

return irs.handleCriticalIncident(incident)

case "HIGH":

return irs.handleHighSeverityIncident(incident)

case "MEDIUM":

return irs.handleMediumSeverityIncident(incident)

default:

return irs.handleLowSeverityIncident(incident)

}

}



func (irs *IncidentResponseSystem) handleCriticalIncident(incident SecurityIncid

// 즉시 격리 모드 활성화

if err := irs.activateQuarantineMode(); err != nil {

irs.logger.Error("Failed to activate quarantine mode", "error", err)

}

// 모든 대응팀에 즉시 알림

for _, responder := range irs.responders {

go responder.NotifyUrgent(incident)

}
```

```go
// 자동 대응 절차 실행
if  procedure, exists := irs.procedures[incident.Type];  exists && procedure.Aut
return  irs.executeProcedure(procedure, incident)
}
return  nil
}


func (irs *IncidentResponseSystem) activateQuarantineMode() error {
// 외부 연결 차단
// 서명 중단
// 로그 수집 강화
// 백업 생성
irs.logger.Warn("Quarantine mode activated - system isolated")
return  nil
}
```

2. 복구 절차

```go
// 시스템 복구 관리자
type RecoveryManager struct {
backupManager *BackupManager
validator *SystemValidator
```

```go
logger log.Logger
}

func (rm *RecoveryManager) InitiateRecovery(incident SecurityIncident) error {
rm.logger.Info("Initiating system recovery", "incident", incident.ID)
// 1. 시스템 상태 평가
status, err := rm.assessSystemStatus()
if err != nil {
return fmt.Errorf("failed to assess system status: %w", err)
}
// 2. 복구 전략 결정
strategy := rm.determineRecoveryStrategy(status, incident)
// 3. 복구 실행
return rm.executeRecovery(strategy)
}

func (rm *RecoveryManager) executeRecovery(strategy RecoveryStrategy) error {
for _, step := range strategy.Steps {
rm.logger.Info("Executing recovery step", "step", step.Name)
if err := step.Execute(); err != nil {
rm.logger.Error("Recovery step failed", "step", step.Name, "error", err)
if step.Critical {
return fmt.Errorf("critical recovery step failed: %w", err)
```

```
}
}
}
// 복구 후 검증
return rm.validator.ValidateRecovery()
}

// 백업 및 복원 시스템
type BackupManager struct {
backupPath string
retentionDays int
encryption EncryptionManager
}

func (bm *BackupManager) CreateEmergencyBackup() error {
timestamp := time.Now().Format("20060102-150405")
backupDir := filepath.Join(bm.backupPath, "emergency", timestamp)
if err := os.MkdirAll(backupDir, 0700); err != nil {
return err
}
// 중요 파일들 백업
filesToBackup := []string{
"config/priv_validator_key.json",
```

```go
"config/node_key.json",
"data/priv_validator_state.json",
"config/config.toml",
"config/genesis.json",
}
for _, file := range filesToBackup {
if err := bm.backupFile(file, backupDir); err != nil {
return fmt.Errorf("failed to backup %s: %w", file, err)
}
}
return nil
}

func (bm *BackupManager) RestoreFromBackup(backupPath string) error {
// 백업 무결성 검증
if err := bm.verifyBackupIntegrity(backupPath); err != nil {
return fmt.Errorf("backup integrity check failed: %w", err)
}
// 현재 상태 백업 (롤백용)
if err := bm.CreateEmergencyBackup(); err != nil {
return fmt.Errorf("failed to create rollback backup: %w", err)
}
// 백업에서 복원
```

```
return  bm.restoreFiles(backupPath)

}
```

이러한 보안 모범사례를 통해 CometBFT 기반 블록체인 네트워크의 보안을 크게 향상시킬 수 있습니다. 중요한 것은 이러한 보안 조치들을 단발성으로 적용하는 것이 아니라, 지속적인 모니터링과 개선을 통해 보안 수준을 유지하고 발전시키는 것입니다.